# XTRA: Unifying Cache Coherence and Concurrency Control for Distributed Transactions in a CXL Pod

Zhijun Yang, Yu Hua*, Ming Zhang, Menglei Chen, Xumin Chen, Aoyang Tong
Huazhong University of Science and Technology

## Abstract

Distributed transactions on CXL memory pools allow hosts to directly access shared data, bypassing message-based coordination. However, the limited cross-host hardware cache coherence (HCC) in CXL hinders synchronization for consistency and concurrency control across large memory pools. To overcome this limitation, we present XTRA, a novel synchronization architecture that efficiently scales C**XL** distributed **TRA**nsactions. XTRA decouples the synchronization state from individual data records, consolidating it into a compact directory within the HCC region, while keeping data in the large non-coherent pool. Distributed transactions leverage this decoupled directory to coordinate concurrent access across the entire shared pool. To ensure efficient and consistent access, XTRA unifies cache coherence with concurrency control by exploiting their semantic redundancy, directly repurposing transactional conflict detection to guarantee cache freshness lazily upon read. Furthermore, by using software mutual exclusion to manage the directory, XTRA's decoupled synchronization generalizes to commodity CXL memory without HCC, while preserving upper-layer protocols. Extensive evaluations demonstrate that XTRA improves throughput by up to 15.6× over state-of-the-art systems and reduces P50/P99 latency by up to 4.0×/283.9×.

***CCS Concepts:*** • **Computer systems organization** → **Distributed architectures**; • **Software and its engineering** → *Concurrency control.*

***Keywords:*** CXL, distributed transactions, cache coherence, concurrency control

**ACM Reference Format:**
Zhijun Yang, Yu Hua, Ming Zhang, Menglei Chen, Xumin Chen, Aoyang Tong. 2026. XTRA: Unifying Cache Coherence and Concurrency Control for Distributed Transactions in a CXL Pod. In *Symposium on Operating Systems Principles (SOSP '26), September 29–October 2, 2026, Prague, Czech Republic.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3830418.3843888



*SOSP '26, September 29–October 2, 2026, Prague, Czech Republic*

ACM ISBN 979-8-4007-2585-2/26/09
https://doi.org/10.1145/3830418.3843888

## 1 Introduction

Modern datacenters are generally constrained by the rigid physical boundaries of monolithic servers, which isolate memory and compute resources at the hardware node level. Consequently, unused resources become stranded within individual machines, leading to severe cluster-wide fragmentation and low utilization [1, 28, 35, 48, 51]. While distributed systems can leverage resources across multiple hosts by partitioning data and execution, the latency of network-based coordination often becomes the performance bottleneck.

To eliminate this network bottleneck, Compute Express Link (CXL) offers a new hardware substrate that breaks the physical boundaries of individual servers [3, 16, 60, 73]. In a CXL pod, multiple hosts are directly attached to a shared, TB-scale memory pool via PCIe, enabling highly concurrent accesses and scalable resource sharing beyond a single machine. These hosts use standard load/store instructions to access CXL memory at cacheline granularity. Frequently accessed lines can be cached in the local CPU L1/L2/L3 caches to reduce remote access latency. Compared with conventional network-based coordination, a CXL pod provides an opportunity to redesign distributed systems around the shared memory, with lower coordination overheads and a much simpler access model [60, 61].

To ensure consistency and atomicity under concurrent accesses, hosts in a CXL pod use distributed transactions to read and write shared data structures in the CXL memory pool. Each transaction ensures that the host executing it observes a consistent read set and that its write set becomes atomically visible at commit. By hiding the complexity of concurrency control and consistency enforcement, distributed transactions have become a foundational primitive in many systems, including distributed file systems [6], databases [18], and serverless platforms [21]. However, it is nontrivial to design and implement efficient distributed transactions over CXL-backed shared memory due to three key challenges.

***C1: Limited support for cross-host hardware cache coherence (HCC).*** Distributed transactions rely on fine-grained synchronization to detect read-write conflicts and serialize concurrent updates across hosts [12, 55, 67]. However, cross-host HCC is absent in CXL 2.0, and hence updates made by one host are not automatically propagated to others [61, 72]. Shared data may silently become stale in remote CPU caches, and synchronization mechanisms such as locks cannot rely on cache-coherent atomic operations like

*Corresponding author: Yu Hua (csyhua@hust.edu.cn).

Compare-and-Swap (CAS) across hosts [18, 33, 40]. Although CXL 3.0 introduces cross-host HCC, practical implementations are expected to support coherence for only a limited region because the device-side snoop filter has limited tracking capacity [11, 18–20, 60]. Consequently, relying on global HCC for transactional synchronization across a large shared memory pool is impractical and inefficient.

***C2: Prohibitive transactional coordination overheads.*** Tigon [18] supports distributed transactions under limited HCC by adopting the Pasha architecture [19], which we refer to as *on-demand sharing*. It partitions data across hosts' local memory by default and, upon remote access, migrates target records into the shared memory pool, while placing their synchronization metadata in the pool's limited HCC region. This allows distributed transactions to use cache-coherent atomic synchronization (e.g., CAS-based locks) on the migrated data, but reintroduces message-based coordination for data migration on the critical path. Specifically, when the target record is absent from the HCC region, the transaction needs to stall, request migration from the owner host, and wait for both data movement and metadata installation before proceeding [18]. Transactional atomicity further amplifies this cost: waiting for one migrated record prolongs the holding time of other locks in the same transaction, thereby widening the contention window and degrading system performance.

***C3: Limited performance of CXL shared memory.*** Compared with local DRAM, CXL memory incurs over 2× the access latency [31, 60], thus slowing shared-data access and increasing transaction latency. Moreover, CXL bandwidth is constrained by PCIe links, thereby limiting transaction throughput under high access concurrency.

Simultaneously resolving these challenges is difficult. To address **C2**, a CXL pod needs to support a *shared-everything* architecture, where any host can directly access any shared data in the memory pool without relying on data partitioning across hosts or message-based coordination for remote accesses. Moreover, to address **C3**, hosts should reduce CXL accesses by caching frequently accessed shared data in local CPU caches, which requires cross-host cache coherence to prevent stale cachelines from violating transactional consistency. However, under the conventional design assumption that each record carries its own fine-grained synchronization state, achieving both goals would require transactional concurrency control (**C2**) and cache coherence (**C3**) across the entire shared pool. This is impractical because CXL hardware cannot scale HCC beyond a limited region (**C1**).

To resolve this tension, we observe that the conventional design's reliance on per-record synchronization state is unnecessary and inefficient. Specifically, although any record may require synchronization over time, the *number* of records participating in active concurrent synchronization at any given moment is limited. Thus, a bounded amount of synchronization state is sufficient and can be dynamically associated with active records across the entire memory pool.

Based on this insight, we present XTRA to efficiently scale distributed transactions in HCC-constrained CXL pods. XTRA adopts a *shared-everything* architecture where shared data resides in a large non-coherent pool, while synchronization state is decoupled from individual data records and consolidated into a compact *Coordination Directory* (CD). Each shared object is deterministically mapped to a CD slot whose lock and version state serves as a synchronization proxy for the object under XTRA's optimistic concurrency control (OCC). In CXL 3.0, XTRA places the CD in the limited HCC region and uses atomic CAS instructions for synchronization. As a result, limited HCC no longer bounds the amount of protected data, but only the amount of active synchronization. This enables a small coherent control plane to coordinate accesses to TB-scale shared data. In CXL 2.0 without HCC, XTRA manages the CD with software mutual exclusion, preserving generality across CXL generations.

XTRA further enables hosts to cache shared data in local CPU caches and DRAM for low-latency access. Because shared data resides in the non-coherent pool, remote updates can silently render local cached copies stale, while eagerly broadcasting invalidation messages to other hosts on each update would reintroduce substantial overhead [23, 29, 53]. Our key insight is that cache coherence and transactional OCC encode overlapping validity semantics: if a transaction validates that a shared object's version is unchanged, its cached copy is also fresh. XTRA exploits this redundancy to unify cache coherence with OCC, using transaction-level version validation to detect and invalidate stale local cached copies lazily on read, rather than eagerly invalidating remote caches on write. This unified coherence avoids HCC dependence while preserving correct and efficient local caching.

In summary, this paper makes the following contributions:

- We present XTRA, a shared-everything distributed transaction system for CXL pods with limited HCC, enabled by decoupling synchronization from data to provide scalable concurrency control over large CXL memory pools.
- We introduce a unified coherence mechanism that exploits the semantic redundancy between cache coherence and OCC, enabling efficient local caching for shared data in CXL memory through lazy on-read cache invalidation.
- We implement and generalize XTRA across CXL 3.0 and non-coherent CXL 2.0, and demonstrate up to 15.6× higher throughput with 4.0×/283.9× lower P50/P99 latency than state-of-the-art systems. The source code of XTRA is publicly available at https://github.com/cszjyang/xtra.

## 2 Background and Motivation

### 2.1 Cross-Host Shared Memory in a CXL Pod

Compute Express Link (CXL) builds on PCIe infrastructure and provides cache, memory, and I/O protocols for communication between processors and attached devices. Specifically, the CXL.mem protocol allows a host CPU to map device

memory into its physical address space and access it via ordinary load/store instructions. A CXL pod extends this model by connecting multiple hosts to a shared CXL memory pool [18, 57, 72, 73]. Recent systems support up to 16 hosts and 8 TB of pooled memory [60]. Compared with prior RDMA-based distributed shared memory [32, 39, 45], CXL can provide 3–10× lower access latency [61]. Moreover, to exploit locality and improve CPU pipeline efficiency, recently accessed data in CXL memory can be cached in CPU caches at cacheline granularity. Consequently, a CXL pod enables distributed systems to replace message-based remote access with direct shared-memory reads and writes, reducing coordination overheads and simplifying programming.

However, the shared-memory abstraction provided by a CXL pod is incomplete. CXL 2.0 supports memory pooling, but does *not* provide hardware cache coherence across hosts [61, 73]. A line cached by one host is hence not automatically invalidated when another host updates the same line. As a result, a host may silently observe stale data from its local CPU caches. Beyond stale reads, the lack of HCC risks data corruption during concurrent updates: CPUs write back to CXL memory at cacheline granularity, so concurrent writebacks from different hosts updating distinct bytes in the same cacheline can blindly overwrite one another. More importantly, standard synchronization primitives like Compare-and-Swap (CAS) fail across hosts, since they require a coherent memory view and cross-host atomicity.

CXL 3.0 partially addresses this limitation by introducing device-mediated cross-host coherence. The device-side snoop filter [11] tracks which hosts cache which shared lines and issues back-invalidations on writes, thereby enabling coherent sharing and standard atomic operations within the tracked region. This tracking state grows with the number of hosts and tracked cachelines. However, implemented in limited on-device SRAM, the snoop filter cannot extend this tracking to the full TB-scale memory pool [7, 19, 37]. Consequently, existing schemes and vendor guidance expect hardware coherence to cover only a limited hardware-cache-coherent (HCC) region (e.g., hundreds of MB [15, 18, 20]), leaving the remaining large capacity as non-hardware-cache-coherent (NHCC) memory [18, 20, 40, 60].

### 2.2 Distributed Transactions in a CXL Pod

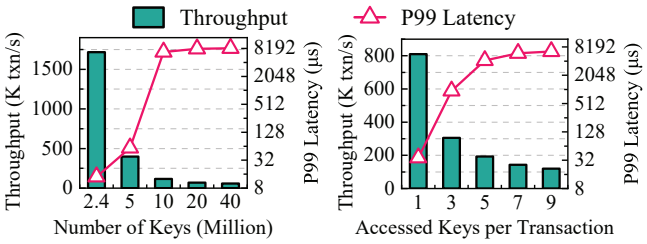
Managing data in a CXL memory pool shared across hosts requires distributed concurrency control and atomic visibility of updates. Distributed transactions naturally provide this abstraction, allowing hosts to concurrently read and update shared data while preserving serializability and atomicity. Each transaction observes a consistent read set, and its write set becomes atomically visible at commit.

To detect read-write conflicts and serialize concurrent updates, conventional transaction designs typically provision fine-grained synchronization metadata (e.g., locks and versions) per data record. This design relies on available synchronization substrates. Specifically, single-node systems [30, 36, 49, 52, 64] leverage cross-core cache coherence to atomically manipulate metadata and guarantee data consistency, while distributed systems coordinate via message passing [10, 22, 41, 47, 65, 66] or one-sided RDMA atomic primitives [12, 43, 55, 56, 67, 68] to acquire locks and fetch data.

However, a CXL pod cannot support these conventional designs across the entire shared pool, as their synchronization state scales with the managed data. While CXL exposes a shared-memory load/store interface, HCC covers only a limited region in CXL 3.0 and is entirely absent in CXL 2.0. Consequently, remote updates can silently render shared data cached in local CPU caches stale, compromising transactional consistency. Furthermore, conventional synchronization mechanisms such as locks cannot rely on cache-coherent atomic primitives like CAS across hosts. Thus, distributed transactions in a CXL pod cannot rely on globally coherent, fine-grained synchronization across the entire shared pool.

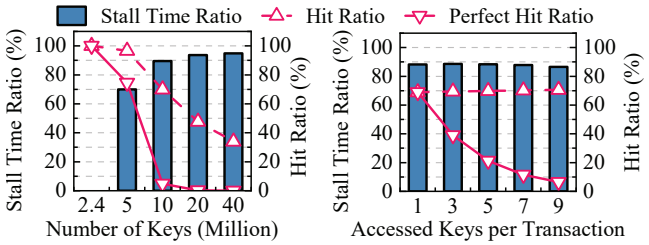
To cope with limited HCC capacity in a CXL 3.0 pod, Tigon [18] adopts an *on-demand sharing* architecture. Instead of placing all shared data in the CXL pool, Tigon partitions ownership across hosts, keeping partitions in local DRAM by default. When a transaction needs a remote object, it first probes the shared HCC region. On a hit, it accesses the shared copy and acquires its lock via CAS for concurrency control. On a miss, the transaction stalls, requesting the remote owner host to migrate the object into the shared pool and install synchronization metadata in the HCC region. If the HCC region is full, the owner evicts objects to make room. Tigon represents an important milestone for distributed transactions in CXL pods by demonstrating how a limited HCC region can support cross-host synchronization for an active subset of records. However, avoiding full-pool coherence comes at the cost of placing remote coordination and data migration on the critical path whenever an HCC miss occurs. Since practical HCC capacity is limited to hundreds of MB [18, 20], far smaller than the TB-scale CXL memory pool, HCC misses become increasingly common as the shared dataset grows. Because of transactional atomicity, each transaction is bottlenecked by its slowest access. Even if all but one of its accesses to remote objects are HCC hits, that *single* miss still triggers remote coordination, forcing the transaction to hold locks already acquired on other objects throughout the long migration delay, thereby blocking concurrent transactions and limiting system throughput.

Evaluating Tigon using a YCSB-style [9] key-value store (KVS) on our CXL testbed (§5.1) quantifies these prohibitive coordination overheads. KV pairs are partitioned across 8 hosts, and each transaction accesses 10 keys following a Zipfian distribution ($\theta = 0.7$). The HCC capacity is limited to 200 MB [18]. Fig. 1a reveals severe performance degradation




**(a)** Varying dataset size **(b)** Varying transaction length

**Figure 1.** Performance of Tigon under a KVS workload.




**(a)** Varying dataset size **(b)** Varying transaction length

**Figure 2.** Root-cause analysis of Tigon via internal metrics.

as the dataset size increases. Tigon achieves peak throughput at 2.4 M keys, where the working set's synchronization metadata fits within the HCC region and distributed transactions rarely incur migration. However, as the dataset grows to 5 M and 40 M keys, the working set exceeds the 200 MB HCC budget. Throughput drops by 4.3× and 30×, while P99 latency rises by 4.2× and 588×, respectively. Fig. 2a explains this degradation. We define a transaction as achieving a *Perfect Hit* when all of its remote key accesses are HCC hits, thus requiring no migration. As the dataset scales up, the per-key hit ratio decreases from 100% to 34%, while the *Perfect Hit* ratio collapses to near zero. Consequently, migration becomes frequent, and stall time accounts for over 90% of transaction latency. We next isolate the impact of transaction length. As shown in Fig. 2b, when we fix the dataset at 10 M keys and vary the number of keys accessed per transaction from 1 to 9, the *Perfect Hit* ratio drops exponentially from 70% to 6%, despite the per-key hit ratio remaining at 70%. This causes stall time to account for over 87% of transaction latency, translating to a 6.8× decrease in throughput and a 182× increase in P99 latency (Fig. 1b).

These results reveal structural limitations of *on-demand sharing*. First, its scalability is constrained by the large capacity gap between the limited HCC region and the much larger shared memory pool. Once the working set exceeds HCC capacity, the coordination and migration overheads dominate transaction execution. Second, the penalty grows disproportionately with transaction length, since longer transactions are increasingly unlikely to achieve a *Perfect Hit* and therefore more likely to stall while waiting for remote coordination. Moreover, ownership partitioning ties the availability of each partition to its owner host. If a host fails, the data in its partition cannot be migrated on demand, preventing other hosts from accessing the partition and blocking distributed transactions that depend on it. Finally, the reliance on CXL 3.0 HCC for atomic locking renders the design inapplicable to widely deployed CXL 2.0 platforms, which provide memory pooling but no HCC.

### 2.3 Toward Shared-Everything CXL Transactions

Overcoming these limitations requires a shared-everything architecture, allowing hosts to directly access shared data in the CXL memory pool without relying on ownership partitioning or critical-path data migration. However, simply placing data in the CXL pool is not enough, since distributed transactions require cross-host concurrency control. Conventional designs provision synchronization metadata per record. Applying these designs in a shared-everything CXL pod requires cache-coherent atomics for this per-record metadata, generating a synchronization footprint that scales with the managed data and eventually exceeds the limited HCC capacity. Moreover, serving every read directly from CXL memory incurs high latency. Caching shared data locally is necessary for performance. However, without full-pool HCC, remote updates can silently render shared data cached in local CPU caches stale, thereby compromising transactional consistency. To address these challenges, XTRA is built on two core design principles:

**Decoupling synchronization state from individual data records.** XTRA observes that while any record may require synchronization over time, the *number* of records participating in active concurrent synchronization at any given moment is limited. Exploiting this property, XTRA abandons conventional per-record metadata. It retains data in the large non-hardware-cache-coherent (NHCC) pool and consolidates synchronization state (locks and versions) into a compact *Coordination Directory* (CD). Distributed transactions can thus leverage this compact CD for optimistic concurrency control (OCC) to coordinate concurrent accesses across the entire TB-scale NHCC shared memory. As a result, limited HCC no longer bounds the amount of protected data, but only the amount of active synchronization.

**Unifying cache coherence with concurrency control.** Caching NHCC-resident data locally requires a coherence protocol. XTRA observes that cache coherence and OCC hinge on the same question: whether a record has changed since its last read. For OCC, this check ensures a transaction's reads remain valid. For coherence, it ensures a cached copy is not stale. Therefore, XTRA unifies the two mechanisms by repurposing OCC version validation as a lazy coherence check. During access, an unchanged CD version guarantees both that the OCC read is valid and the cached copy is fresh for reuse. Otherwise, a version change triggers local cache

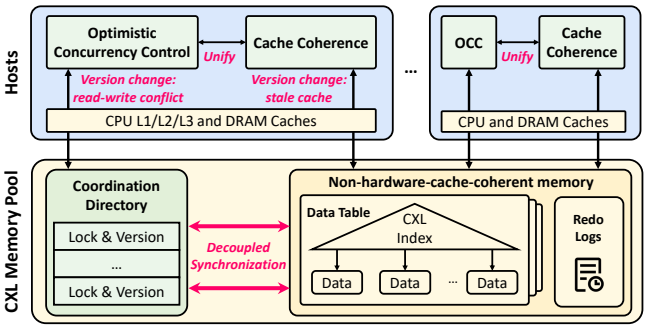



**Figure 3.** Overview of XTRA. CD synchronization uses HCC atomics in CXL 3.0 or software mutual exclusion in CXL 2.0.

invalidation. This design preserves efficient cache coherence without full-pool HCC.

## 3 XTRA Overview

Fig. 3 shows XTRA's overall architecture. The CXL shared memory is organized into two logical regions: a large data-plane NHCC region and a small control-plane Coordination Directory (CD). The NHCC region stores shared data tables and indexes (e.g., $B^+$-trees). Each shared object (e.g., a data record or an index node) is deterministically mapped to a CD slot that stores the lock and version state used for synchronization (§4.1). To ensure consistency, threads across hosts access shared objects through XTRA transactions, which leverage the corresponding CD locks and versions to detect conflicts and serialize commits (§4.2). Committed updates advance the corresponding CD versions. XTRA further reuses these versions as lazy coherence signals, invalidating stale copies in local CPU caches when version validation detects a remote update (§4.3). In CXL 3.0, XTRA places the small CD in the limited HCC region to enable atomic primitives such as CAS. In §4.4, we further generalize the same CD abstraction to CXL 2.0, where HCC is absent, using software mutual exclusion. Finally, XTRA stores redo logs in the shared NHCC region, enabling recovery from the shared pool while preserving transactional atomicity (§4.5).

## 4 XTRA Design

### 4.1 Coordination Directory Structure and Mapping

To scale transactional synchronization beyond the coherence-capacity limit of a CXL pod, XTRA abandons the conventional design of provisioning synchronization metadata for each data object. Instead, XTRA decouples synchronization metadata from individual objects and consolidates it into a compact *Coordination Directory* (CD), while leaving data objects and index nodes in NHCC memory.

**Directory structure.** The CD is a flat array of 64-bit slots. As shown in Fig. 4, each slot encodes the synchronization state for its mapped objects: a 50-bit version counter for optimistic validation and lazy coherence, a 1-bit lock flag, a

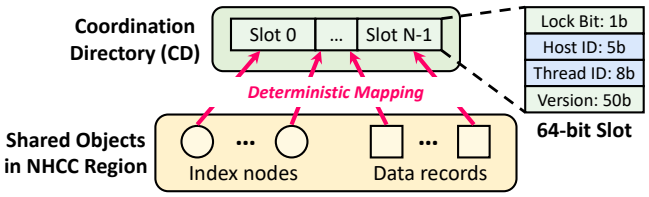



**Figure 4.** Coordination Directory structure and mapping.

5-bit host identifier, and an 8-bit per-host thread identifier. When the lock flag is set, the host/thread pair identifies the lock holder. Currently, these fields support up to 32 hosts and 256 threads per host, and can be adjusted for larger CXL pod configurations.

**Atomic abstraction.** XTRA treats each CD slot as a logically atomic synchronization unit. In CXL 3.0, the CD resides in the limited HCC region, allowing transactions to manipulate directory slots with cross-host coherent operations, such as atomic Compare-and-Swap (CAS) for lock acquisition and coherent loads/stores for version checks and updates. As a result, limited HCC capacity bounds only the volume of concurrent synchronization, rather than the total amount of protected data in the TB-scale pool. In §4.4, we show how the same atomic-slot abstraction extends to CXL 2.0, where HCC is absent, using software mutual exclusion. This abstraction hides hardware differences from the upper layers: both the OCC protocol and the unified coherence protocol operate on atomic directory slots regardless of whether atomicity is provided by HCC or software.

**Deterministic mapping.** For decoupled synchronization to be correct, every host must derive the same CD slot for a given shared object (e.g., a data record or an index node). We observe that, although the CXL memory may be mapped at different virtual addresses on different hosts, an object's offset within the shared pool is invariant once the object is allocated. Prior CXL-pod systems also rely on offset-based addressing to maintain pointer consistency despite differing virtual mappings across hosts [18, 40, 75]. XTRA therefore uses the object's *offset pointer*, i.e., its offset relative to a base memory pointer in the shared CXL pool, as the canonical identifier for directory lookup. Accordingly, XTRA maps each object to a directory slot by hashing its pool offset: $slot = Hash(offset) \bmod N$, where $N$ is the number of CD slots. This mapping is deterministic across hosts, requires only constant-time computation, and does not depend on application-specific semantics.

**Correctness of decoupled synchronization.** The key requirement for any synchronization scheme is *agreement*: all hosts must consult the same synchronization state for the same shared object. Conventional methods satisfy this requirement with a *coupled* design, where each object carries its own synchronization metadata (e.g., an embedded lock and version word). XTRA preserves the same agreement

property with a *decoupled* design: every host deterministically maps an object to the same CD slot, and all synchronization for that object is performed through that slot. As a result, any two accesses that contend on the same object necessarily contend on the same directory slot.

The inherent trade-off of this decoupled design is the possibility of *CD-slot mapping collisions*. When distinct objects map to the same CD slot, they may incur *false contention*, causing concurrent accesses to independent objects to be conservatively serialized. However, because no true conflicts are ever missed, these collisions affect only concurrency, not correctness. Crucially, accepting this trade-off enables XTRA to multiplex scarce HCC capacity across the TB-scale NHCC pool. Rather than provisioning metadata for every stored object, XTRA uses a bounded directory to cover only the objects undergoing *active synchronization*. Consequently, the directory size $N$ is determined by the expected concurrency and available HCC budget, not by the total dataset size. For example, an 8 MB directory provides $10^6$ 64-bit slots. Even in a large pod with 32 hosts and 256 threads per host, if all 8,192 threads concurrently execute transactions that each access a dozen objects, the directory load factor remains below 10%, limiting the likelihood of collision-induced false contention.

### 4.2 Decoupled Optimistic Concurrency Control

XTRA serializes distributed transactions with a *decoupled* optimistic concurrency control (OCC) protocol over the CD. Unlike conventional designs that associate locks and versions with individual objects, decoupled OCC performs conflict detection and commit serialization through the CD slots to which those objects map. OCC is inherently well-suited for this CD-based synchronization because it minimizes the runtime impact of slot collisions. Since OCC reads remain lock-free and depend only on version validation, slot collisions do not introduce blocking on the read path. Furthermore, exclusive locks are acquired only for the write set and deferred until speculative execution completes, thereby confining collision-induced false contention to a brief commit window rather than the full transaction lifetime. As a result, decoupled OCC allows XTRA to use a compact directory to coordinate transactions over a much larger shared pool, while keeping the false contention caused by slot collisions modest. For clarity, we first assume that each access to an NHCC object returns a fresh value. §4.3 later shows how XTRA enforces this property by unifying coherence with OCC validation. As shown in Fig. 5, the transaction lifecycle proceeds in three phases:

**1. Execution.** The transaction executes speculatively and accesses objects in the NHCC region as needed. For each access, the thread first resolves the target object's offset in the shared pool by traversing the shared index for a key lookup, and then reads the corresponding CD slot. If the slot is not locked, it reads the object and records the slot version. If the slot is locked, another transaction may have

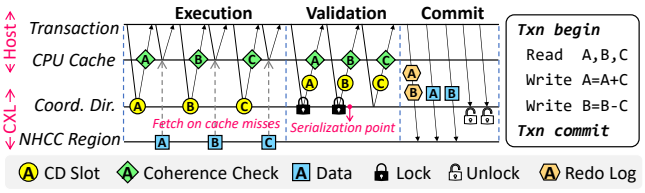



**Figure 5.** The distributed transaction protocol of XTRA.

modified the target object; the current transaction thus aborts and retries to avoid reading inconsistent data and violating serializability. During this phase, XTRA acquires no write locks. Any updates are buffered locally in the transaction's write set, which may overlap with the read set for read-modify-write accesses.

**2. Locking and validation.** After speculative execution completes, the transaction attempts to lock the CD slots corresponding to its write set. Locks are acquired using the CD's atomic-slot abstraction: CAS on CXL 3.0, or the software exclusion mechanism described in §4.4 on CXL 2.0. If any required slot is already locked by another transaction, the lock acquisition fails and the transaction aborts. Once all write locks are acquired, the transaction validates its read set by checking that the mapped slots are unlocked (or locked by itself) and the slot versions still match the versions observed during execution. Any mismatch indicates that the transaction's read set may no longer be consistent, forcing the transaction to release its write locks and abort.

**3. Commit.** After successful locking and validation, the transaction has established its serialization point and can commit. It first writes and flushes the transaction's entire redo log to the shared NHCC log area to enable transaction recovery after a host failure (§4.5). It then applies the buffered updates to the target NHCC objects, issuing cache-line writebacks and ordering fences (e.g., `CLWB` and `SFENCE`) to ensure the updates become visible in shared memory. Finally, the transaction releases each locked slot with a single store that simultaneously clears the lock bit and advances the slot version.

Despite decoupling synchronization from data, XTRA preserves the same safety property as conventional OCC. Consider two transactions, $T_1$ and $T_2$, with a read-write conflict on an object $O$. Because all hosts deterministically map $O$ to the same CD slot, $T_1$ and $T_2$ are forced to coordinate through the same lock and version state. If $T_1$ updates $O$ and commits first, it must hold the mapped slot lock during its update and increment the version upon release. If $T_2$ previously read $O$, its read set is now stale. During its subsequent validation phase, $T_2$ will either encounter the slot as locked (if $T_1$ is still committing) or observe the version mismatch (if $T_1$ has finished). In either case, the conflict is detected, forcing $T_2$ to abort rather than commit inconsistent data. Thus, the CD slot acts as a logical proxy for the object, ensuring that

any successful commit is based on a consistent read set and providing atomic visibility for the write set.

**Handling slot collisions.** Because multiple objects may map to the same CD slot, XTRA needs to handle both inter-transaction and intra-transaction slot collisions. Consider two distinct objects, $O_1$ and $O_2$, that deterministically map to the same slot $S$. Inter-transaction collisions arise when two concurrent transactions access these independent objects, e.g., $T_1$ accesses $O_1$ while $T_2$ accesses $O_2$. If both accesses are reads, neither transaction aborts because OCC reads remain lock-free and do not advance the slot version. However, if either access is a write (e.g., $T_1$ writes to $O_1$), it must lock $S$ and advance the version upon commit. $T_2$ may then observe $S$ as locked or detect a version change during validation, forcing it to conservatively abort. Thus, under OCC, inter-transaction slot collisions may induce false conflicts only when the collision involves a write, affecting performance rather than correctness.

Intra-transaction collisions occur when a single transaction $T$ accesses both $O_1$ and $O_2$. If $T$ writes to both objects, it will attempt to acquire the exclusive lock for $S$ twice. To prevent $T$ from deadlocking with itself during these lock acquisitions, XTRA treats slot locking as reentrant. If $T$ already holds the lock on $S$ for $O_1$, acquiring the lock for $O_2$ succeeds automatically. This reentrancy introduces a visibility risk: if $T$ were to release $S$ immediately after updating $O_1$, this would inadvertently expose an intermediate state before $T$ updates $O_2$. XTRA's commit protocol naturally avoids this vulnerability, since locks are released only after the entire write set is updated and made globally visible. This design efficiently achieves reentrant locking for slot collisions without the overhead of per-slot reentrancy counters.

**Handling index operations.** Because index nodes reside in NHCC memory and may be concurrently modified, XTRA coordinates index operations using an optimistic crabbing protocol [27] adapted to CD-based decoupled synchronization. Index modifications (e.g., inserts, deletes, and updates) acquire exclusive locks on the CD slots of all index nodes they modify. Like OCC reads, index searches traverse the index without locking, but validate each pointer dereference against the source node's CD-slot state. Concretely, after following a pointer from a parent to a child, the search thread re-checks the parent's CD slot. If the slot is locked or its version has changed, a concurrent update may have modified the parent's contents, potentially invalidating the traversal path and forcing the lookup to restart from the root. This protocol ensures that hosts locate needed objects based on a consistent index traversal, while keeping searches lock-free.

### 4.3 Transaction-Driven Unified Cache Coherence

To reduce access latency, CXL allows host CPUs to automatically cache frequently accessed shared-memory cachelines. However, because XTRA places shared data in the NHCC region to bypass hardware scalability limits, these cached copies lack cross-host hardware coherence. Consequently, a remote update may silently leave stale data in another host's CPU caches. If a transaction reuses such a stale cacheline, it may observe an inconsistent read set and violate serializability. Naively enforcing software coherence would require tracking active sharers and eagerly broadcasting invalidation messages to remote hosts upon every commit [23, 29, 53], reintroducing prohibitive coordination overheads.

To efficiently ensure cache coherence for NHCC-resident data, XTRA exploits a key insight that there is a fundamental semantic redundancy between cache coherence and OCC. Both mechanisms hinge on answering the same question, i.e., whether an object has changed since it was last read. In OCC, this check determines whether a transaction's read set remains valid; in cache coherence, it determines whether a locally cached copy remains fresh. Thus, if a CD slot version is validated as unchanged for OCC, the same validation can also serve as a cache-freshness signal for the corresponding CPU-cached copy. Leveraging this redundancy, XTRA reuses transactional version checks to ensure cache coherence lazily, only when data is read, thereby eliminating the need for an eager invalidation protocol.

However, repurposing transactional validation for cache coherence reveals two scope mismatches. First, a **temporal mismatch**: OCC validation is *transient*, scoped strictly to a single transaction's lifetime, whereas cache coherence requires *continuous* validity across the system lifetime. Second, a **spatial mismatch**: while transactions provide *logical isolation* across threads, the CPU last-level cache (LLC) is *physically shared* among all threads on a host. Even when a thread accesses an object for the first time, it may inadvertently consume a stale cacheline populated by a different local thread's prior access. Consequently, transaction-scoped, thread-local OCC validation alone is insufficient to guarantee coherence for NHCC data cached in the CPU caches.

**Shadow Directory.** To bridge the mismatches above and realize unified coherence, XTRA introduces a *Shadow Directory* (SD), a host-local shared registry that transactions consult to maintain coherence for CXL-resident data cached in local CPU caches. First, the SD addresses the *temporal mismatch* by tracking the most recently observed value of each CD slot's version ($V_{slot}$) using a corresponding local *shadow version* ($V_{shadow}$), thereby allowing coherence knowledge to persist across transactions rather than disappearing after OCC validation. Second, it addresses the *spatial mismatch* by sharing object-level coherence state among local threads, thereby allowing one thread to determine whether another thread on the same host has already established coherence for a target object under the current slot version.

As shown in Fig. 6, the SD resides in host-local DRAM and structurally mirrors the shared CD: the $i$-th shadow slot corresponds to the $i$-th CD slot. Each shadow slot comprises an 8-byte header and a compact array of *tracking entries*.

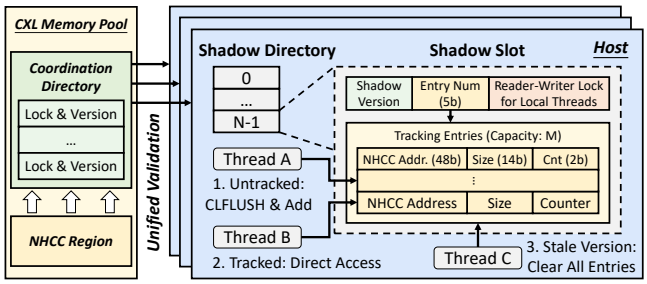



**Figure 6.** Unified cache coherence via Shadow Directories.

The header stores $V_{shadow}$, a lock field for inter-thread synchronization, and the number of valid tracking entries. Each 8-byte tracking entry records one NHCC object by storing its 48-bit address, its 14-bit size in cacheline units, and a 2-bit access counter used for lightweight frequency-based replacement when the array is full. The array size $M$ is configurable to match workload locality and memory budgets. We currently set $M$=7 so that each shadow slot fits within one 64-byte cacheline, enabling efficient per-access lookup.

**Lazy invalidation protocol.** XTRA avoids eager invalidation on remote updates because such invalidation is expensive and often wasteful if the updated data is not subsequently consumed by the local host. Instead, it ensures coherence *lazily* upon read, guaranteeing that transactions never consume stale data. The key observation is that the transaction read path already fetches the CD slot version for OCC validation. XTRA therefore piggybacks its coherence check on this existing version validation (Fig. 5), eliminating dedicated coherence messages. The protocol handles three distinct scenarios:

**1. Access with tracked coherence.** When a transaction accesses an NHCC object, it first reads the object's mapped CD slot from CXL. If the CD slot is locked, the transaction aborts. Otherwise, the thread compares the slot's current version $V_{slot}$ against the shadow version $V_{shadow}$ stored in the corresponding SD slot. If $V_{slot} = V_{shadow}$, no remote update has occurred on this slot. The thread then scans the SD tracking entries under a read lock, which is efficient since the shadow slot fits within a single cacheline. If the object is already tracked (Thread B in Fig. 6), its CPU-cached copy is guaranteed fresh. The thread safely accesses it via direct memory loads and updates the entry access counter (if $< 3$) using a lock-free CAS.

**2. Sanitization for untracked objects.** If $V_{slot} = V_{shadow}$ but the target object is not currently tracked in the SD (Thread A in Fig. 6), XTRA cannot trust the host-local CPU caches, since stale cachelines may exist due to prior accesses or hardware prefetching. The thread therefore performs *sanitization-on-entry* by first issuing `CLFLUSH` on the object's cachelines to invalidate any potentially stale cached copies, and then installing a tracking entry for the object in the SD. The invalidation ensures that the next load fetches fresh data from NHCC memory. This sanitization-on-entry is performed under the shadow-slot write lock, which safely excludes concurrent local accesses to the same shadow slot. Once the installed entry becomes visible, subsequent local threads can follow Step 1 and safely reuse the cached copy without sanitization, provided $V_{slot} = V_{shadow}$.

**3. Staleness resolution.** If $V_{slot} > V_{shadow}$, remote transactions have updated objects mapped to this slot since the host last established coherence under $V_{shadow}$, potentially leaving local cached copies stale. The thread then updates $V_{shadow}$ to $V_{slot}$, clears all tracking entries under the shadow-slot write lock, and falls back to Step 2. This ensures that subsequent local threads cannot treat any object mapped to the slot as coherence-validated under the new version before re-establishing tracking via sanitization-on-entry.

**Crash consistency.** The SD requires neither persistence nor recovery. If a host fails, both the SD in DRAM and the host-local CPU caches are lost together. After restart, the host begins with an empty SD and no residual cached state, so coherence can be re-established from scratch without additional recovery logic.

**Optional host-local DRAM cache.** While CPU caches offer the lowest latency, their limited capacity can still lead to hardware evictions of hot NHCC objects, forcing later transactions to refetch the cachelines from the CXL memory pool. To mitigate these capacity misses, XTRA allows hosts to optionally cache hot objects in host-local DRAM. This DRAM cache requires no separate coherence protocol. It reuses the same version-based lazy invalidation as CPU caches by verifying the cached version against the mapped CD slot's current version $V_{slot}$ on the OCC path. This tiered hierarchy allows CPU caches to handle the hottest working set while host DRAM absorbs capacity misses that would otherwise incur CXL latency. As a pure performance optimization, the DRAM cache can be selectively enabled based on workload locality and update intensity.

### 4.4 Generalization to Non-Coherent CXL 2.0

XTRA leverages CXL 3.0 HCC and cross-host atomic CAS to efficiently realize atomic CD slots, whereas CXL 2.0 provides neither cross-host coherence nor global atomics. To generalize XTRA to CXL 2.0 without altering the upper-layer decoupled OCC or unified coherence protocols, XTRA preserves the same atomic-slot abstraction in software.

**Software-managed atomic CD slots.** In CXL 2.0, XTRA places the CD in the NHCC region and adapts CD-slot access in two ways. First, because NHCC-resident CD slots may be stale in host-local CPU caches, XTRA explicitly invalidates any cached copy before reading a slot (e.g., via `CLFLUSH`), ensuring that lock and version checks observe the latest shared state. Second, hardware CAS-based lock acquisition is replaced by load/store-based software mutual exclusion.

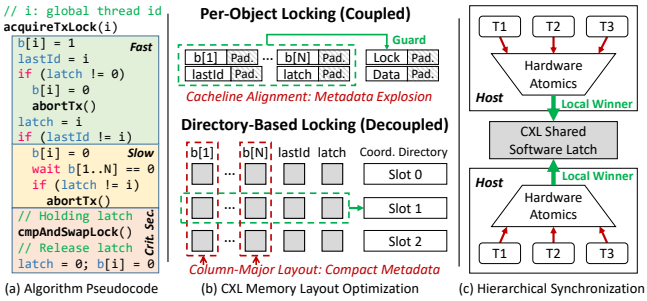



**Figure 7.** Managing the CD via software mutual exclusion.

As shown in Fig. 7a, each CD slot is associated with a software latch based on Lamport's fast mutual exclusion algorithm [25]. Competing threads first serialize through this latch. The thread entering the critical section acquires the slot lock if it is free, and then releases the latch. The latch protects only the slot's lock-state transition, thereby providing the atomic lock-acquisition step required by XTRA's OCC protocol. Like CD slots, the latch metadata resides in NHCC memory and hence requires cache invalidation before reads.

**Why decoupling makes software exclusion feasible.** Load/store-based mutual exclusion among $N$ participants has an $\Omega(N)$ space lower bound [8], as exemplified by the flags $b[1..N]$ in Fig. 7a. As discussed in §2.1, CPUs write back to CXL memory at cacheline granularity. To prevent cross-host blind overwrites, flags written by different hosts must reside in separate 64 B cachelines. Consequently, the metadata footprint of a single software latch grows linearly with the number of hosts, exceeding 512 B in an 8-host CXL pod. In a conventional *coupled* design, provisioning this bloated metadata for every protected data object would lead to prohibitive memory amplification (Fig. 7b, top).

XTRA makes software exclusion practical through two architectural optimizations (Fig. 7b, bottom). First, because synchronization is decoupled from data, latch metadata is provisioned per CD slot rather than per object. Second, XTRA adopts a column-major layout for the mutex flags to avoid per-flag cacheline padding while preserving write isolation across hosts. Specifically, instead of storing all flags of one latch together, XTRA packs flags updated by the same host across adjacent CD slots. Because all flags within such a cacheline are written by only one host, cross-host blind overwrites are avoided without requiring one cacheline per flag. As a result, the metadata footprint of software exclusion remains compact and does not scale with the size of the protected dataset.

**Efficient synchronization.** As shown in Fig. 7a, Lamport's algorithm minimizes shared-memory accesses on its uncontended fast path [25], but its contended slow path requires scanning $N$ participant flags, which is expensive over high-latency CXL links. XTRA reduces this cost with a hierarchical synchronization mechanism (Fig. 7c). Instead of exposing all threads to direct inter-host contention, threads on the same host first compete using low-latency local atomics. Only the local winner proceeds to contend on the inter-host software latch. This hierarchy reduces the effective participant count on the CXL path to at most one per host, substantially decreasing the number of remote flag checks under contention.

XTRA further minimizes latch overhead through its OCC protocol. For slot reads, such as version validation and lock checking, transactions directly inspect CD slots without entering the software latch. For slot writes, the latch is needed only for lock acquisition, when a transaction attempts to set the lock bit of a CD slot. Because the lock holder already possesses exclusive write ownership of the slot, it can directly release the slot lock without entering the software latch again, using a single CXL write that clears the lock bit and advances the version.

Through these optimizations, XTRA supports efficient transactional synchronization on commodity CXL 2.0 platforms without requiring custom hardware support.

### 4.5 Fault Tolerance

**Failure model.** XTRA assumes a fail-stop model, where hosts may crash independently at any time (e.g., OS panic or power loss) but do not exhibit Byzantine behavior. Following prior work on CXL pods [40, 69, 75], we assume that the shared CXL memory pool remains available to surviving hosts after a host failure. Failures of the memory pool itself are outside XTRA's current failure model and are discussed in §6. Under this model, a host crash may interrupt its in-flight transactions, leaving write locks unreleased in the CD and data updates partially applied in the CXL memory. Unreleased locks can permanently block surviving hosts from accessing the data, and partial updates can violate atomicity and consistency by leaving data in intermediate states.

**Logging and recovery.** XTRA uses write-ahead redo logging to preserve transactional atomicity under host failures. During commit, a transaction writes and flushes redo log records to its assigned buffer in the shared NHCC region. Each log record contains the operation type, table ID, key, and value required to replay the write. A transaction applies updates only after its redo log is complete in shared memory.

We assume a standard membership service for host-failure detection [75]. When a host failure is detected, surviving hosts can initiate recovery without waiting for the failed host to reboot, since both the data and the redo logs remain globally available in shared memory. Recovery proceeds in two steps. First, recovery scans the CD to identify locks held by the failed host. Second, recovery inspects the failed host's log buffers to determine whether the redo log for each transaction associated with these locks is complete. If a transaction's redo log is complete, recovery rolls the transaction forward by replaying its logged updates and then releasing its locks. If the redo log is incomplete, the failed transaction could not have applied any data update, so

recovery aborts the transaction by simply releasing its locks. As a result, recovery needs to inspect only the compact CD and the failed host's active log buffers, avoiding host-reboot latency and restoring system availability quickly.

# 5 Performance Evaluation

## 5.1 Experimental Setup

**Testbed.** Our evaluation machine has two Intel Xeon Gold 6530 CPUs and 256 GB of local DRAM, and is attached to a 128 GB CXL 2.0 memory device. Since CXL 3.0 platforms with cross-host HCC are not yet commercially available, we follow state-of-the-art methodologies [5, 18, 75] and emulate an 8-host CXL pod using 8 virtual machines (VMs, each with 4 CPU cores and 32 GB of local memory) that share the same CXL memory device.

**Benchmarks.** We use three representative OLTP workloads: a YCSB-style [9] key-value store (KVS), SmallBank [2], and TPCC [4]. The KVS consists of a single table with 4-byte keys and 128-byte values. It supports flexible workload configuration, including transaction length, access skew, and per-access write ratio. SmallBank and TPCC use their standard transaction mixes. SmallBank represents each account with two 12-byte records across two tables (*Savings* and *Checking*). Its transactions are short and write-intensive: each transaction accesses at most three records, and 85% of transactions are read-write. TPCC consists of nine tables with record sizes ranging from 16 to 672 bytes. Overall, 92% of its transactions are read-write, and we vary the cross-warehouse access probabilities of `NewOrder` and `Payment` to control the fraction of distributed transactions. For each benchmark, we define three dataset scales (**S/M/L**) in Table 1. The **S**-scale dataset sizes for KVS and TPCC match those used in Tigon [18].

**Comparisons.** We evaluate two variants: **XTRA** targets CXL 3.0 pods by leveraging the limited HCC region for atomic CD management (§4.1), whereas **XTRA-NHCC** targets non-coherent CXL 2.0 pods by managing the CD via software mutual exclusion (§4.4). We compare them against three distributed transaction systems: Tigon [18], Sundial [65], and Motor [67]. **Tigon** [18] is a state-of-the-art CXL 3.0 transaction system using *on-demand sharing*. As presented in §2.2, it partitions data in host-local DRAM and migrates remotely requested records into shared CXL memory, utilizing CXL 3.0 atomics to perform two-phase locking (2PL) and avoid two-phase commit (2PC). **Sundial-CXL** adapts Sundial's [65] shared-nothing protocol to CXL 3.0 pods using Tigon's optimized implementation [18]. It employs a hybrid OCC/2PL protocol based on dynamic timestamp leasing and incorporates Tigon's on-demand sharing for hot data, allowing it to leverage CXL 3.0 atomics and avoid 2PC. **Motor** [67] is a shared-everything RDMA baseline. Its multi-version concurrency control (MVCC) protocol is built on one-sided RDMA primitives and requires RDMA CAS across the full memory pool for synchronization. Because a CXL pod with limited HCC cannot meet this requirement, we evaluate Motor in its native RDMA environment.

| Workload | S (small) | M (medium) | L (large) |
|---|---|---|---|
| KVS (#records) | 2.4M | 10M | 40M |
| SmallBank (#accounts) | 1.2M | 5M | 20M |
| TPCC (#warehouses) | 24 | 240 | 960 |

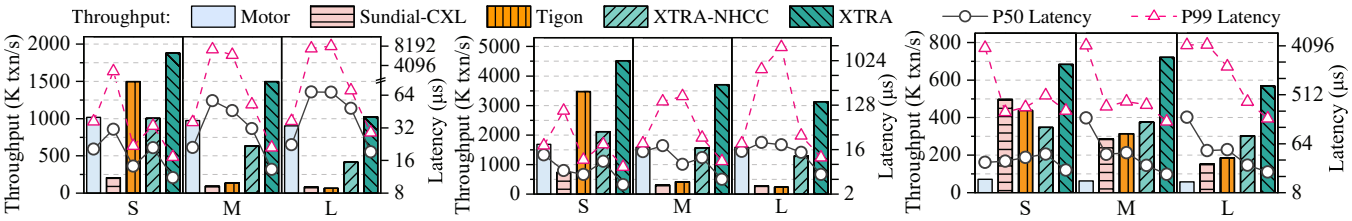
**Table 1.** Default dataset scales (total across 8 hosts).

**Configurations.** We run the CXL-based systems on the emulated CXL pod, warming up for 10 s and measuring performance for 30 s. Consistent with Tigon [18], the pod's HCC capacity is limited to 200 MB. Accordingly, we configure a 200 MB CD for both XTRA (fitting within the HCC) and XTRA-NHCC (residing in the NHCC region). Inter-VM cache coherence is provided by the physical machine and should be faster than real inter-host HCC when it becomes available. To ensure fair comparisons, we align configurations as follows: (1) We replace the CXL baselines' default SSD-backed logging with XTRA's CXL logging to eliminate orthogonal SSD-induced overheads. (2) The CXL baselines require one CPU core per VM for migration message passing. Although XTRA does not require such a core, we idle one core per VM for XTRA to match the baselines' CPU budget. (3) As in Tigon [18], all CXL-based systems place warehouse-private TPCC tables in host-local DRAM because these tables are not shared across hosts. For shared tables in the CXL pool, XTRA's host DRAM cache capacity matches Tigon's per-host local partition size. We run Motor on four physical machines (one compute node and three memory nodes [67]), each equipped with a ConnectX-5 RDMA NIC connected to a 100 Gbps InfiniBand switch. The compute node uses the same thread count as the CXL systems.

## 5.2 End-to-End Performance across Scales

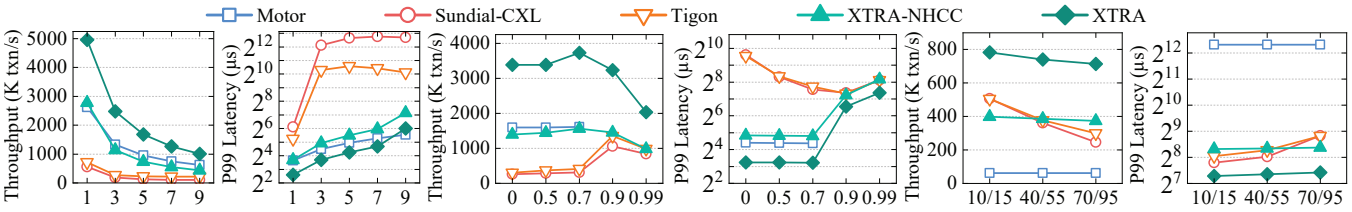
We evaluate the systems across the S, M, and L dataset scales defined in Table 1. In KVS, each transaction accesses 10 records with a 5% write ratio per access, making 40% of transactions read-write. The Zipfian skew parameter $\theta$ for KVS and SmallBank is set to 0.7 [18, 67]. For TPCC, we set the cross-warehouse access probabilities of `NewOrder` and `Payment` to 60% and 90%, respectively, following Tigon [18].

Fig. 8 demonstrates that Motor maintains stable performance across all dataset scales because RDMA provides direct data access and synchronization across the entire memory pool. In contrast, on-demand sharing systems (Tigon and Sundial-CXL) suffer severe performance collapses at larger scales. Tigon achieves competitive performance at the S scale because the small working set's synchronization metadata fits entirely within the limited HCC region, thereby ensuring that almost all remote accesses are HCC hits. However, at the M and L scales, the working set exceeds HCC capacity, forcing transactions to frequently stall and




**(a)** KVS **(b)** SmallBank **(c)** TPCC

**Figure 8.** System throughput and latency (P50 and P99) across three dataset scales (Small, Medium, and Large).




**(a)** KVS: varying transaction length **(b)** SmallBank: varying skewness (Zipfian $\theta$) **(c)** TPCC: varying cross-warehouse percentage

**Figure 9.** Sensitivity of throughput and P99 latency to variations in workload characteristics at the M scale.

wait for remote records to be migrated into the shared pool via message-based coordination, as analyzed in §2.2. Consequently, in KVS and SmallBank, Tigon's throughput drops by 8.3–22.8×, while its P50 and P99 latencies increase by 1.6–4.8× and 19.4–374.5×, respectively, causing it to fall behind RDMA-based Motor. Tigon degrades less in TPCC because most accesses target warehouse-private tables in host-local DRAM and thus largely bypass the shared pool. Sundial-CXL slightly outperforms Tigon in TPCC-S because its dynamic timestamp leasing effectively reduces transactional conflicts. However, in KVS-S and SmallBank-S, Sundial-CXL underperforms Tigon in throughput by 7.2× and 4.7×, respectively, because its dynamic protocol requires a larger per-record metadata footprint, prematurely exhausting HCC capacity and triggering frequent migrations.

XTRA sustains high performance across all scales, delivering 1.3–15.6× higher throughput and 1.3–4.0×/1.2–283.9× lower P50/P99 latency than Tigon, while outperforming Motor by 1.1–11.5× in throughput and 1.3–25.5× in P99 latency. Enabled by the CD-based decoupled OCC, XTRA's shared-everything architecture achieves scalable transactional synchronization across the entire shared pool via direct CXL memory accesses, overcoming HCC capacity constraints and eliminating the need for message-based coordination. As the dataset scale increases from S to L, XTRA's throughput degrades gracefully by 16.9–45.6% because the larger working set causes more CPU cache misses and thus more frequent CXL memory accesses. Targeting non-coherent CXL 2.0, XTRA-NHCC outperforms Tigon by 1.2–6.4× in throughput at the M and L scales, demonstrating that decoupled synchronization remains scalable without hardware coherence. At the S scale, XTRA-NHCC underperforms Tigon by 20.8–39.0% in throughput because managing its non-coherent CD requires explicit cache invalidation and software mutual exclusion over high-latency CXL memory, whereas Tigon uses hardware-accelerated CXL 3.0 atomics enabled by HCC.

### 5.3 Sensitivity to Workload Characteristics

Fixing the dataset at the M scale, we evaluate system sensitivity to variations in workload characteristics. Across all configurations shown in Fig. 9, XTRA outperforms the best baselines, delivering 1.5–2.4× higher throughput and up to 2.6× lower P99 latency.

In KVS, the write ratio per access is set to 50%. As shown in Fig. 9a, increasing the transaction length (the number of accessed keys per transaction) from 1 to 3 reduces Tigon's throughput by 2.6× and disproportionately increases its P99 latency by 33.2×. This is because longer transactions exponentially decrease the *Perfect Hit* ratio (i.e., the probability that all of a transaction's remote key accesses are HCC hits). As analyzed in §2.2, a single HCC miss stalls a Tigon transaction awaiting remote migration while retaining already-acquired locks, severely bottlenecking concurrency. As the length increases from 1 to 9, XTRA's throughput also decreases by 4.9× due to more CXL memory accesses in longer transactions. Compared with CXL systems, the RDMA-based Motor exhibits more modest degradation as transaction length increases, because RDMA allows batching multiple asynchronous accesses within a single round-trip time, whereas CXL requires individual memory accesses.

Fig. 9b demonstrates that, in SmallBank, increasing the Zipfian skew $\theta$ to 0.7 initially boosts XTRA's throughput by

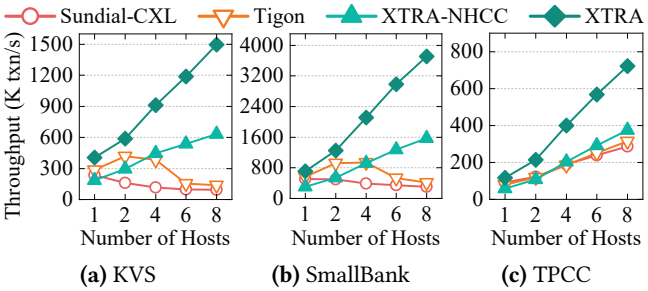



**Figure 10.** Throughput under varying numbers of hosts.

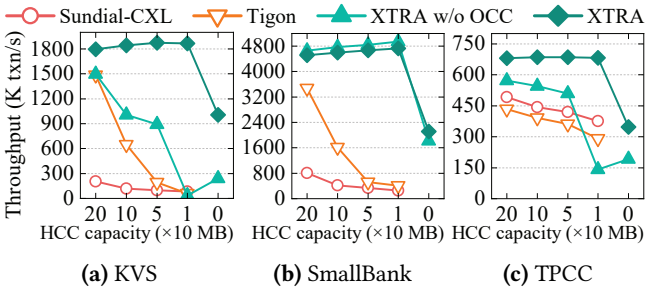



**Figure 11.** Throughput under varying HCC capacities.

10% via improved CPU cache locality. For Tigon and Sundial-CXL, high skew ($\theta = 0.9$) increases the *Perfect Hit* ratio because the highly concentrated hot set's metadata fits within the limited HCC region, yielding a 3.3× throughput increase. However, extreme skewness ($\theta = 0.99$) eventually degrades the performance of all systems by exacerbating write-write conflicts. Motor fails to complete the benchmark beyond $\theta = 0.9$ due to observed livelocks under severe contention.

In Fig. 9c, we increase the cross-warehouse access probabilities of `NewOrder`/`Payment` transactions in TPCC from the default 10%/15% to 70%/95%. Increasing these probabilities degrades both Tigon's throughput and P99 latency by 1.7×, with more pronounced degradation for Sundial-CXL. Higher cross-warehouse access probabilities require more data to be dynamically migrated into shared CXL memory, increasing pressure on the limited HCC region, thereby triggering severe HCC misses and message-based coordination overheads. In contrast, XTRA and XTRA-NHCC maintain stable performance as these probabilities increase. Their decoupled synchronization efficiently coordinates transactional accesses across the entire shared pool without requiring the synchronization footprint to scale with the protected dataset.

### 5.4 System Scalability

To evaluate system scalability, we increase the number of hosts from 1 to 8 while keeping the data volume per host constant, with the aggregate dataset reaching the M scale at 8 hosts. The remaining setup follows §5.2. As Fig. 10 shows, Tigon and Sundial-CXL scale poorly in KVS and SmallBank due to the 200 MB HCC limit. When scaling from 1 to 2 hosts, the synchronization metadata for Tigon's shared working set still fits within the HCC region, allowing Tigon to avoid remote migrations and increase throughput by 46.8%. However, beyond 2 hosts, the aggregate working set exceeds HCC capacity, triggering frequent migrations and causing Tigon's throughput to fall by 3.1×. Sundial-CXL degrades even earlier because its larger metadata footprint exhausts the HCC sooner. In TPCC, the baselines increase their throughput by up to 4.0× from 1 to 8 hosts because over 90% of accesses are warehouse-private and remain local, thereby largely bypassing CXL HCC constraints. In contrast, XTRA and XTRA-NHCC consistently scale across all workloads, increasing their throughput by 3.7–6.1× and outperforming the baselines by up to 10.9× at 8 hosts. By using a compact CD to decouple synchronization state from the protected dataset, they avoid the HCC-capacity bottleneck encountered by the baselines as the aggregate dataset grows with the number of hosts. XTRA-NHCC remains slower than XTRA because it relies on software-based mutual exclusion to manage the CD, whereas XTRA leverages hardware-accelerated CXL 3.0 HCC atomics for efficient synchronization.

### 5.5 Impact of HCC Capacity and Decoupled OCC

Fixing the dataset at the S scale, we evaluate system robustness by shrinking the HCC capacity from 200 MB down to 0 MB. XTRA's HCC-resident CD is sized to match the available HCC budget. The remaining setup follows §5.2. As Fig. 11 shows, Tigon performs competitively when the synchronization metadata for its shared working set fits within the 200 MB HCC. However, when the HCC budget falls below 100 MB, the metadata for the working sets of Tigon and Sundial-CXL no longer fits within the coherent region, triggering frequent message-based migrations and causing throughput to collapse. At 0 MB HCC, neither system can operate because both require HCC atomics. In contrast, XTRA consistently sustains high throughput down to 10 MB of HCC, outperforming Tigon by 1.2–31.5× across different workloads and HCC capacities. Counterintuitively, reducing the CD size can slightly improve XTRA's performance. This is because a smaller CD occupies less space in CPU caches, leaving more cache space for NHCC data, thereby increasing cache hits and reducing accesses to CXL memory. Even at 0 MB HCC, XTRA remains operational by falling back to XTRA-NHCC, which places a 200 MB CD in the NHCC region. This fallback incurs a 43.9–53.0% throughput drop relative to XTRA at 10 MB HCC because hardware atomics are replaced by software mutual exclusion.

To isolate the impact of CD size on concurrency control, we evaluate **XTRA w/o OCC** (using 2PL). In KVS and TPCC,

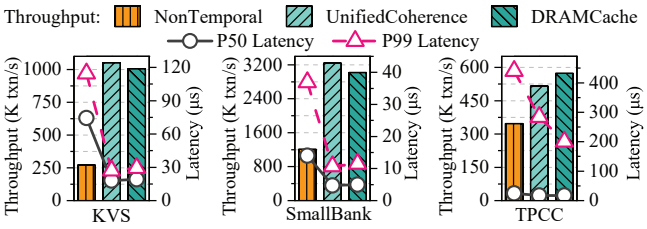



**Figure 12.** Effectiveness of unified cache coherence.

XTRA w/o OCC degrades severely as the CD shrinks. Unlike OCC, 2PL acquires read and write locks during execution, holding them longer and heavily exacerbating false contention from slot collisions in smaller CDs. XTRA's decoupled OCC minimizes this false contention via lock-free reads and brief validation-phase write-locking, increasing its speedup over 2PL from 1.2× to 50.6× as the HCC capacity shrinks to 10 MB. At 0 MB HCC, even with a 200 MB NHCC-resident CD, XTRA w/o OCC remains substantially slower than XTRA-NHCC in KVS and TPCC. This gap arises because 2PL requires shared read-lock acquisitions that incur costly serialization through software latches over high-latency CXL links, whereas OCC avoids this serialization overhead with lock-free reads. However, OCC is not universally optimal: in the highly write-intensive SmallBank, OCC slightly underperforms 2PL, since read-write conflicts trigger frequent OCC validation failures and costly aborts, which 2PL effectively avoids via pessimistic read locks.

### 5.6 Unified Cache Coherence

Fixing the dataset at the L scale, we evaluate the effectiveness of XTRA's unified cache coherence mechanism. The remaining setup follows §5.2. While XTRA's CD enables decoupled concurrency control to scale across the entire pool, reading NHCC data requires additional mechanisms to guarantee freshness without hardware coherence. In Fig. 12, the *NonTemporal* baseline bypasses the CPU cache using non-temporal loads/stores to directly access CXL memory, preventing stale reads but incurring high CXL access latencies. In contrast, XTRA's unified cache coherence repurposes OCC version validation to achieve lazy coherence upon reads. This minimizes coherence overhead and enables local CPU cache hits, effectively reducing expensive CXL accesses. Consequently, across different workloads, the unified coherence design improves throughput by 1.5×–3.9× over the baseline, while reducing P50 and P99 latencies by 1.4×–4.1× and 1.5×–4.2×, respectively. The gains vary across workloads, with KVS showing the largest improvements. SmallBank benefits less because its write-intensive workload causes more frequent version changes and cache invalidations, reducing opportunities to reuse cached data. TPCC sees the smallest gain because most accesses target warehouse-private tables in local DRAM and thus largely bypass the shared CXL pool.

Additionally, XTRA supports an optional *DRAM cache* by extending the same lazy coherence protocol to the host-local DRAM tier. However, this software cache incurs management and data-copy overheads while also increasing the CPU cache footprint. As a result, the DRAM cache is not always beneficial, negatively impacting performance in KVS and SmallBank. In TPCC, the strong access locality and complex access patterns allow the DRAM cache to effectively absorb CPU cache capacity misses, yielding an 11.2% throughput improvement and a 29.6% P99 latency reduction.

## 6 Discussion

**Supporting Datasets Beyond CXL Capacity.** XTRA focuses on transactional synchronization and coherence for shared data resident in the TB-scale CXL memory pool. Our current prototype and evaluation assume that this shared data fits within the pool, but XTRA does not require all application data to reside there. Host-private data can remain in local DRAM, as in our TPCC configuration, which keeps warehouse-private tables in local DRAM while placing shared tables in the CXL pool (§5.1). For applications whose shared dataset exceeds CXL memory capacity, XTRA can be combined with a separate caching [63] or tiering layer that manages data residency between the CXL memory pool and a larger backing tier. Objects resident in the shared CXL memory pool retain XTRA's existing direct-access path with CD-based concurrency control and unified coherence, while capacity misses may require data movement from the backing tier. Importantly, XTRA's CD remains bounded because it is provisioned for active synchronization rather than for every object in the backing dataset. Whereas Tigon's on-demand sharing triggers data migration on misses in the much smaller HCC region, the relevant capacity boundary for XTRA is the entire CXL memory pool. Designing and evaluating a complete capacity-management policy is outside XTRA's current scope and remains future work.

**Memory-Pool Fault Tolerance.** XTRA's current recovery mechanism (§4.5) handles host failures while assuming that the shared CXL memory pool remains available. One direction for tolerating pool-level data loss is to use checkpoint-and-log recovery techniques [70]. XTRA could periodically checkpoint shared data to a failure-independent domain and persist subsequent redo logs there. After a pool failure, the system could reconstruct a consistent shared-data state by restoring the latest checkpoint and replaying subsequent complete redo logs. Alternatively, XTRA could replicate shared data across CXL memory pools in independent failure domains so that a surviving replica remains available after a pool failure [67]. Once a consistent shared-data state is available, XTRA could reinitialize the CD and re-establish coherence from scratch by clearing each host's SD and invalidating all host-local cached copies before resuming transactions. These extensions would change the

durability and pool-recovery substrate, but need not fundamentally alter XTRA's decoupled OCC or unified coherence protocols. Designing and evaluating such pool-level fault tolerance remains future work.

## 7 Related Work

**CXL memory disaggregation and sharing.** With CXL 1.0 enabling single-host memory expansion, recent work has extensively explored memory tiering [13, 26, 35, 42, 46, 50, 58, 59, 71, 74] to overcome single-node capacity walls. Furthermore, CXL 2.0 introduces multi-host, non-coherent memory pooling [14, 28]. Recent distributed systems exploit this substrate for fast RPCs [33, 34], serverless computing [5, 17], memory allocation [40], I/O pooling [72], and databases [61]. However, lacking cross-host HCC, these CXL 2.0 systems typically manage concurrent read-write accesses by falling back to message passing or relying on custom FPGA-based near-memory processing to enforce atomic operations. To provide standard shared-memory semantics, CXL 3.0 introduces cross-host HCC [11, 44]. However, the physical capacity of the hardware snoop filter strictly limits the size of this coherent region [15, 18, 54]. Consequently, scaling consistent data sharing and concurrent accesses across CXL 3.0 pods remains challenging.

**Real CXL-pod use cases.** Recent systems demonstrate concrete use cases in which multiple hosts directly access shared data in CXL memory pools. Duhu [37] uses a prototype CXL memory pool with Ray [38], allowing workers across hosts to directly access shared intermediate objects without first copying them into host-local memory, thereby reducing data-movement overheads. Beluga [60] builds a shared memory pool for GPU clusters using a commercial CXL 2.0 switch and applies it to KVCache management in vLLM [24], enabling GPUs across servers to share and reuse KVCache data. PolarCXLMem [61] integrates CXL shared memory into PolarDB-MP [62], allowing multiple database primaries to directly access shared buffer pages in the CXL pool, while using distributed page locks to coordinate concurrent accesses and software cache invalidation to maintain coherence. Together, these systems demonstrate that CXL pods can serve as a practical shared-memory substrate for diverse multi-host workloads. XTRA targets CXL-pod workloads in which multiple hosts concurrently access shared data and require transactional consistency and concurrency control.

**Distributed transactions on CXL pods.** Conventional transaction systems [12, 41, 49, 55, 67] typically maintain fine-grained synchronization metadata for individual records, causing synchronization state to scale with the managed data. Tigon [18] and CtXnL [54] inherit this design assumption when leveraging cross-host HCC in CXL 3.0 pods to coordinate transactions across hosts. Since HCC is limited, CtXnL relies on custom FPGA-based extensions at the CXL memory endpoint to alter the coherence protocol, making it incompatible with unmodified commodity hardware and precluding direct empirical comparison. Targeting standard hardware, Tigon copes with limited HCC via on-demand sharing based on data migrations. Despite their different deployment strategies, both systems share a fundamental limitation: their per-record synchronization footprint scales with the shared working set and eventually exceeds the limited HCC capacity, forcing costly coordination that limits scalability. In contrast, XTRA abandons per-record metadata and consolidates synchronization state into a bounded CD to scale concurrency control across the entire CXL memory pool. This decoupled synchronization not only overcomes HCC constraints but also readily generalizes to non-coherent CXL 2.0 environments. Furthermore, XTRA unifies cache coherence with OCC, leveraging lazy on-read invalidation to ensure consistent and low-latency data sharing.

## 8 Conclusion

CXL creates opportunities to redesign distributed systems around shared memory, but its limited cross-host HCC fundamentally bottlenecks concurrent and consistent access for distributed transactions. To address this, XTRA decouples synchronization state from data and consolidates it into a bounded Coordination Directory to scale concurrency control across the entire pool, while efficiently ensuring cache coherence by piggybacking lazy cache invalidation onto optimistic read validation. This cohesive approach pushes highly concurrent and consistent transactions in a CXL pod beyond HCC constraints, enabling XTRA to significantly outperform state-of-the-art systems. Beyond distributed transactions, XTRA's decoupled synchronization architecture provides a scalable blueprint for other distributed systems in CXL pods.

## Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant No. 62125202 and U22B2022. We sincerely thank our shepherd and the anonymous reviewers for their constructive suggestions and feedback. We also thank Yibo Huang at the University of Texas at Austin for valuable discussions.

## References

[1] 2019. Google Cluster Trace 2019. https://github.com/google/cluster-data/blob/master/ClusterData2019.md.
[2] 2022. SmallBank Benchmark. https://hstore.cs.brown.edu/documentation/deployment/benchmarks/smallbank/.
[3] 2024. Compute Express Link. https://computeexpresslink.org/.
[4] 2024. TPC-C Benchmark. http://www.tpc.org/tpcc/.
[5] Chloe Alverti, Stratos Psomadakis, Burak Ocalan, Shashwat Jaiswal, Tianyin Xu, and Josep Torrellas. 2025. *CXLfork: Fast Remote Fork over CXL Fabrics.* Association for Computing Machinery, New York, NY, USA, 210–226. https://doi.org/10.1145/3676641.3715988
[6] Thomas E. Anderson, Marco Canini, Jongyul Kim, Dejan Kostić, Youngjin Kwon, Simon Peter, Waleed Reda, Henry N. Schuh, and

Emmett Witchel. 2020. Assise: Performance and Availability via Client-local NVM in a Distributed File System. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 1011–1027. https://www.usenix.org/conference/osdi20/presentation/anderson

[7] Daniel S. Berger. 2024. Realistic Expectations for CXL Memory Pools. Keynote at the 2nd Workshop on Disruptive Memory Systems (DIMES'24). https://dimes.ws/2024/

[8] J.E. Burns and N.A. Lynch. 1993. Bounds on Shared Memory for Mutual Exclusion. *Inf. Comput.* 107, 2 (Dec. 1993), 171–184. https://doi.org/10.1006/inco.1993.1065

[9] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM symposium on Cloud computing*. 143–154.

[10] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, JJ Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. 2012. Spanner: Google's Globally-Distributed Database. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. USENIX Association, Hollywood, CA, 261–264. https://www.usenix.org/conference/osdi12/technical-sessions/presentation/corbett

[11] CXL Consortium. 2022. *Compute Express Link (CXL) Specification, Revision 3.0.* Compute Express Link Consortium. https://computeexpresslink.org/wp-content/uploads/2024/02/CXL3.0-Specification.pdf Available at Compute Express Link Consortium website.

[12] Aleksandar Dragojević, Dushyanth Narayanan, Edmund B. Nightingale, Matthew Renzelmann, Alex Shamis, Anirudh Badam, and Miguel Castro. 2015. No compromises: distributed transactions with consistency, availability, and performance. In *Proceedings of the 25th Symposium on Operating Systems Principles* (Monterey, California) *(SOSP '15)*. Association for Computing Machinery, New York, NY, USA, 54–70. https://doi.org/10.1145/2815400.2815425

[13] Padmapriya Duraisamy, Wei Xu, Scott Hare, Ravi Rajwar, David Culler, Zhiyi Xu, Jianing Fan, Christopher Kennelly, Bill McCloskey, Danijela Mijailovic, Brian Morris, Chiranjit Mukherjee, Jingliang Ren, Greg Thelen, Paul Turner, Carlos Villavieja, Parthasarathy Ranganathan, and Amin Vahdat. 2023. Towards an Adaptable Systems Architecture for Memory Tiering at Warehouse-Scale. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 727–741. https://doi.org/10.1145/3582016.3582031

[14] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. 2022. Direct Access, High-Performance Memory Disaggregation with DirectCXL. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 287–294. https://www.usenix.org/conference/atc22/presentation/gouk

[15] Jiyu Hu, Seokjoo Cho, Landon Johnson, Kiran Hombal, Shreesha Gopalakrishna Bhat, Marcos K. Aguilera, Ramnatthan Alagappan, and Aishwarya Ganesan. 2026. MEGALON: Efficient Data Sharing for Partly-Coherent CXL Memory. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26)*. USENIX Association, Seattle, WA, USA. https://www.usenix.org/conference/osdi26/presentation/hu-jiyu

[16] Yu Hua, Xue Liu, and Ion Stoica. 2026. The Golden Rule of Big Memory: Persistence Is Not Harmful. *Commun. ACM* 69, 5 (May 2026), 51–55. https://doi.org/10.1145/3769003

[17] Jialiang Huang, MingXing Zhang, Teng Ma, Zheng Liu, Sixing Lin, Kang Chen, Jinlei Jiang, Xia Liao, Yingdi Shan, Ning Zhang, Mengting Lu, Tao Ma, Haifeng Gong, and YongWei Wu. 2024. TrEnv: Transparently Share Serverless Execution Environments Across Different Functions and Nodes. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) *(SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 421–437. https://doi.org/10.1145/3694715.3695967

[18] Yibo Huang, Haowei Chen, Newton Ni, Yan Sun, Vijay Chidambaram, Dixin Tang, and Emmett Witchel. 2025. Tigon: A Distributed Database for a CXL Pod. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA.

[19] Yibo Huang, Newton Ni, Vijay Chidambaram, Emmett Witchel, and Dixin Tang. 2025. Pasha: An Efficient, Scalable Database Architecture for CXL Pods. In *15th Conference on Innovative Data Systems Research, CIDR 2025, Amsterdam, The Netherlands, January 19-22, 2025*. www.cidrdb.org.

[20] Sunita Jain, Nagaradhesh Yeleswarapu, Hasan Al Maruf, and Rita Gupta. 2024. Memory Sharing with CXL: Hardware and Software Design Approaches. arXiv:2404.03245 [cs.ET] https://arxiv.org/abs/2404.03245

[21] Zhipeng Jia and Emmett Witchel. 2021. Boki: Stateful Serverless Computing with Shared Logs. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (Virtual Event, Germany) *(SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 691–707. https://doi.org/10.1145/3477132.3483541

[22] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2016. FaSST: Fast, Scalable and Simple Distributed Transactions with Two-Sided (RDMA) Datagram RPCs. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, Savannah, GA, 185–201. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/kalia

[23] Peter Keleher, Alan L. Cox, Sandhya Dwarkadas, and Willy Zwaenepoel. 1994. Tread Marks: Distributed Shared Memory on Standard Workstations and Operating Systems. In *USENIX Winter 1994 Technical Conference (USENIX Winter 1994 Technical Conference)*. USENIX Association, San Francisco, CA. https://www.usenix.org/conference/usenix-winter-1994-technical-conference/tread-marks-distributed-shared-memory-standard

[24] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. https://doi.org/10.1145/3600006.3613165

[25] Leslie Lamport. 1987. A fast mutual exclusion algorithm. *ACM Trans. Comput. Syst.* 5, 1 (Jan. 1987), 1–11. https://doi.org/10.1145/7351.7352

[26] Taehyung Lee, Sumit Kumar Monga, Changwoo Min, and Young Ik Eom. 2023. MEMTIS: Efficient Memory Tiering with Dynamic Page Classification and Page Size Determination. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 17–34. https://doi.org/10.1145/3600006.3613167

[27] Viktor Leis, Florian Scheibner, Alfons Kemper, and Thomas Neumann. 2016. The ART of practical synchronization. In *Proceedings of the 12th International Workshop on Data Management on New Hardware* (San Francisco, California) *(DaMoN '16)*. Association for Computing Machinery, New York, NY, USA, Article 3, 8 pages. https://doi.org/10.1145/2933349.2933352

[28] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating*

*Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 574–587. https://doi.org/10.1145/3575693.3578835

[29] Kai Li and Paul Hudak. 1989. Memory coherence in shared virtual memory systems. *ACM Trans. Comput. Syst.* 7, 4 (Nov. 1989), 321–359. https://doi.org/10.1145/75104.75105

[30] Hyeontaek Lim, Michael Kaminsky, and David G. Andersen. 2017. Cicada: Dependably Fast Multi-Core In-Memory Transactions. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) *(SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 21–35. https://doi.org/10.1145/3035918.3064015

[31] Jinshu Liu, Hamid Hadian, Hanchen Xu, Daniel S Berger, and Huaicheng Li. 2024. Dissecting CXL Memory Performance at Scale: Analysis, Modeling, and Optimization. *arXiv preprint arXiv:2409.14317* (2024).

[32] Xuchuan Luo, Jiacheng Shen, Pengfei Zuo, Xin Wang, Michael R Lyu, and Yangfan Zhou. 2024. CHIME: A Cache-Efficient and High-Performance Hybrid Index on Disaggregated Memory. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 110–126.

[33] Teng Ma, Zheng Liu, Chengkun Wei, Jialiang Huang, Youwei Zhuo, Haoyu Li, Ning Zhang, Yijin Guan, Dimin Niu, Mingxing Zhang, and Tao Ma. 2024. HydraRPC: RPC in the CXL Era. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 387–395. https://www.usenix.org/conference/atc24/presentation/ma

[34] Suyash Mahar, Ehsan Hajyjasini, Seungjin Lee, Zifeng Zhang, Mingyao Shen, and Steven Swanson. 2024. Telepathic Datacenters: Fast RPCs using Shared CXL Memory. arXiv:2408.11325 [cs.DC] https://arxiv.org/abs/2408.11325

[35] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. 2023. Tpp: Transparent page placement for cxl-enabled tiered-memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 742–755.

[36] Syed Akbar Mehdi, Deukyeon Hwang, Simon Peter, and Lorenzo Alvisi. 2023. ScaleDB: A Scalable, Asynchronous In-Memory Database. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 361–376. https://www.usenix.org/conference/osdi23/presentation/mehdi

[37] Qiutong Men, Tao Wang, Jongryool Kim, Hane (Stella) Yie, Emmanuel Amaro, Marcos K. Aguilera, and Aurojit Panda. 2026. Duhu: Shared Disaggregated Memory for Distributed Data Processing Frameworks. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Seattle, WA, 921–938. https://www.usenix.org/conference/osdi26/presentation/men

[38] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 561–577. https://www.usenix.org/conference/osdi18/presentation/moritz

[39] Antoine Murat, Clément Burgelin, Athanasios Xygkis, Igor Zablotchi, Marcos Kawazoe Aguilera, and Rachid Guerraoui. 2024. SWARM: Replicating Shared Disaggregated-Memory Data in No Time. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 24–45.

[40] Newton Ni, Yan Sun, Zhiting Zhu, and Emmett Witchel. 2026. Cxlalloc: Safe and Efficient Memory Allocation for a CXL Pod. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)* (Pittsburgh, PA, USA) *(ASPLOS '26)*. ACM, New York, NY, USA, 18. https://doi.org/10.1145/3779212.3790149

[41] Daniel Peng and Frank Dabek. 2010. Large-scale incremental processing using distributed transactions and notifications. In *Proceedings of the 9th USENIX Conference on Operating Systems Design and Implementation* (Vancouver, BC, Canada) *(OSDI'10)*. USENIX Association, USA, 251–264.

[42] Zhenlin Qi, Shengan Zheng, Ying Huang, Yifeng Hui, Bowen Zhang, Linpeng Huang, and Hong Mei. 2025. Chrono: Meticulous Hotness Measurement and Flexible Page Migration for Memory Tiering. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 835–853. https://doi.org/10.1145/3689031.3717462

[43] Alex Shamis, Matthew Renzelmann, Stanko Novakovic, Georgios Chatzopoulos, Aleksandar Dragojević, Dushyanth Narayanan, and Miguel Castro. 2019. Fast General Distributed Transactions with Opacity. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 433–448. https://doi.org/10.1145/3299869.3300069

[44] Debendra Das Sharma, Robert Blankenship, and Daniel S Berger. 2023. An introduction to the compute express link (cxl) interconnect. *arXiv preprint arXiv:2306.11227* (2023).

[45] Jiacheng Shen, Pengfei Zuo, Xuchuan Luo, Yuxin Su, Jiazhen Gu, Hao Feng, Yangfan Zhou, and Michael R Lyu. 2023. Ditto: An elastic and adaptive memory-disaggregated caching system. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 675–691.

[46] Yan Sun, Jongyul Kim, Zeduo Yu, Jiyuan Zhang, Siyuan Chai, Michael Jaemin Kim, Hwayong Nam, Jaehyun Park, Eojin Na, Yifan Yuan, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2025. *M5: Mastering Page Migration and Memory Management for CXL-based Tiered Memory Systems*. Association for Computing Machinery, New York, NY, USA, 604–621. https://doi.org/10.1145/3676641.3711999

[47] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J. Abadi. 2012. Calvin: fast distributed transactions for partitioned database systems. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data* (Scottsdale, Arizona, USA) *(SIGMOD '12)*. Association for Computing Machinery, New York, NY, USA, 1–12. https://doi.org/10.1145/2213836.2213838

[48] Muhammad Tirmazi, Adam Barker, Nan Deng, Md E. Haque, Zhijing Gene Qin, Steven Hand, Mor Harchol-Balter, and John Wilkes. 2020. Borg: the next generation. In *EuroSys '20: Fifteenth EuroSys Conference 2020, Heraklion, Greece, April 27-30, 2020*. ACM, 30:1–30:14.

[49] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy transactions in multicore in-memory databases *(SOSP '13)*. Association for Computing Machinery, New York, NY, USA, 18–32. https://doi.org/10.1145/2517349.2522713

[50] Midhul Vuppalapati and Rachit Agarwal. 2024. Tiered Memory Management: Access Latency is the Key!. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) *(SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 79–94. https://doi.org/10.1145/3694715.3695968

[51] Midhul Vuppalapati, Justin Miron, Rachit Agarwal, Dan Truong, Ashish Motivala, and Thierry Cruanes. 2020. Building An Elastic Query Engine on Disaggregated Storage. In *17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020, Santa Clara, CA, USA, February 25-27, 2020*. USENIX Association, 449–462.

[52] Jiachen Wang, Ding Ding, Huan Wang, Conrad Christensen, Zhaoguo Wang, Haibo Chen, and Jinyang Li. 2021. Polyjuice: High-Performance Transactions via Learned Concurrency Control. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 198–216. https://www.usenix.org/conference/osdi21/presentation/wang-jiachen

[53] Qing Wang, Youyou Lu, Erci Xu, Junru Li, Youmin Chen, and Jiwu Shu. 2021. Concordia: Distributed Shared Memory with In-Network Cache Coherence. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, 277–292. https://www.usenix.org/conference/fast21/presentation/wang

[54] Zhao Wang, Yiqi Chen, Cong Li, Yijin Guan, Dimin Niu, Tianchan Guan, Zhaoyang Du, Xingda Wei, and Guangyu Sun. 2025. CtXnL: A Software-Hardware Co-designed Solution for Efficient CXL-Based Transaction Processing. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 192–209.

[55] Xingda Wei, Zhiyuan Dong, Rong Chen, and Haibo Chen. 2018. Deconstructing RDMA-enabled Distributed Transactions: Hybrid is Better!. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 233–251. https://www.usenix.org/conference/osdi18/presentation/wei

[56] Xingda Wei, Jiaxin Shi, Yanzhe Chen, Rong Chen, and Haibo Chen. 2015. Fast in-memory transaction processing using RDMA and HTM. In *Proceedings of the 25th Symposium on Operating Systems Principles* (Monterey, California) *(SOSP '15)*. Association for Computing Machinery, New York, NY, USA, 87–104. https://doi.org/10.1145/2815400.2815419

[57] Emmett Witchel. 2024. Challenges and Opportunities for Systems Using CXL Memory *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 2. https://doi.org/10.1145/3620666.3655590

[58] Lingfeng Xiang, Zhen Lin, Weishu Deng, Hui Lu, Jia Rao, Yifan Yuan, and Ren Wang. 2024. NOMAD: non-exclusive memory tiering via transactional page migration. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 2, 17 pages.

[59] Dong Xu, Junhee Ryu, Kwangsik Shin, Pengfei Su, and Dong Li. 2024. FlexMem: Adaptive Page Profiling and Migration for Tiered Memory. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 817–833. https://www.usenix.org/conference/atc24/presentation/xu-dong

[60] Xinjun Yang, Qingda Hu, Junru Li, Feifei Li, Yicong Zhu, Yuqi Zhou, Qiuru Lin, Jian Dai, Yang Kong, Jiayu Zhang, Guoqiang Xu, and Qiang Liu. 2025. Beluga: A CXL-Based Memory Architecture for Scalable and Efficient LLM KVCache Management. arXiv:2511.20172 [cs.DC] https://arxiv.org/abs/2511.20172

[61] Xinjun Yang, Yingqiang Zhang, Hao Chen, Feifei Li, Gerry Fan, Yang Kong, Bo Wang, Jing Fang, Yuhui Wang, Tao Huang, Wenpu Hu, Jim Kao, and Jianping Jiang. 2025. Unlocking the Potential of CXL for Disaggregated Memory in Cloud-Native Databases. In *Companion of the 2025 International Conference on Management of Data* (Berlin, Germany) *(SIGMOD/PODS '25)*. Association for Computing Machinery, New York, NY, USA, 689–702. https://doi.org/10.1145/3722212.3724460

[62] Xinjun Yang, Yingqiang Zhang, Hao Chen, Feifei Li, Bo Wang, Jing Fang, Chuan Sun, and Yuhui Wang. 2024. PolarDB-MP: A Multi-Primary Cloud-Native Database via Disaggregated Shared Memory. In *Companion of the 2024 International Conference on Management of Data* (Santiago AA, Chile) *(SIGMOD '24)*. Association for Computing Machinery, New York, NY, USA, 295–308. https://doi.org/10.1145/3626246.3653377

[63] Zhijun Yang, Yu Hua, Ming Zhang, Menglei Chen, and Yixiao Wang. 2026. FORGE: Mitigating Synchronization Amplification for Memory-Disaggregated Caching Systems. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Seattle, WA, 977–995. https://www.usenix.org/conference/osdi26/presentation/yang-zhijun

[64] Xiangyao Yu, Andrew Pavlo, Daniel Sanchez, and Srinivas Devadas. 2016. TicToc: Time Traveling Optimistic Concurrency Control. In *Proceedings of the 2016 International Conference on Management of Data* (San Francisco, California, USA) *(SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 1629–1642. https://doi.org/10.1145/2882903.2882935

[65] Xiangyao Yu, Yu Xia, Andrew Pavlo, Daniel Sanchez, Larry Rudolph, and Srinivas Devadas. 2018. Sundial: harmonizing concurrency control and caching in a distributed OLTP database management system. *Proc. VLDB Endow.* 11, 10 (June 2018), 1289–1302. https://doi.org/10.14778/3231751.3231763

[66] Irene Zhang, Naveen Kr. Sharma, Adriana Szekeres, Arvind Krishnamurthy, and Dan R. K. Ports. 2015. Building consistent transactions with inconsistent replication. In *Proceedings of the 25th Symposium on Operating Systems Principles* (Monterey, California) *(SOSP '15)*. Association for Computing Machinery, New York, NY, USA, 263–278. https://doi.org/10.1145/2815400.2815404

[67] Ming Zhang, Yu Hua, and Zhijun Yang. 2024. Motor: Enabling {Multi-Versioning} for Distributed Transactions on Disaggregated Memory. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 801–819.

[68] Ming Zhang, Yu Hua, Pengfei Zuo, and Lurong Liu. 2022. FORD: Fast One-sided RDMA-based Distributed Transactions for Disaggregated Persistent Memory. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*. USENIX Association, Santa Clara, CA, 51–68. https://www.usenix.org/conference/fast22/presentation/zhang-ming

[69] Mingxing Zhang, Teng Ma, Jinqi Hua, Zheng Liu, Kang Chen, Ning Ding, Fan Du, Jinlei Jiang, Tao Ma, and Yongwei Wu. 2023. Partial Failure Resilient Memory Management System for (CXL-based) Distributed Shared Memory. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 658–674. https://doi.org/10.1145/3600006.3613135

[70] Wenting Zheng, Stephen Tu, Eddie Kohler, and Barbara Liskov. 2014. Fast Databases with Fast Durability and Recovery Through Multicore Parallelism. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, 465–477. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/zheng_wenting

[71] Yuhong Zhong, Daniel S Berger, Carl Waldspurger, Ishwar Agarwal, Rajat Agarwal, Frank Hady, Karthik Kumar, Mark D Hill, Mosharaf Chowdhury, and Asaf Cidon. 2024. Managing Memory Tiers with CXL in Virtualized Environments. In *Symposium on Operating Systems Design and Implementation*.

[72] Yuhong Zhong, Daniel S. Berger, Pantea Zardoshti, Enrique Saurez, Jacob Nelson, Dan R. K. Ports, Antonis Psistakis, Joshua Fried, and Asaf Cidon. 2025. Oasis: Pooling PCIe Devices Over CXL to Boost Utilization *(SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 101–119. https://doi.org/10.1145/3731569.3764812

[73] Yuhong Zhong, Fiodar Kazhamiaka, Pantea Zardoshti, Shuwei Teng, Rodrigo Fonseca, Mark D. Hill, and Daniel S. Berger. 2026. Octopus: Enhancing CXL Memory Pods via Sparse Topology. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, Renton, WA, 1303–1322. https://www.usenix.org/conference/nsdi26/presentation/zhong

[74] Zhe Zhou, Yiqi Chen, Tao Zhang, Yang Wang, Ran Shu, Shuotao Xu, Peng Cheng, Lei Qu, Yongqiang Xiong, Jie Zhang, and Guangyu Sun. 2024. NeoMem: Hardware/Software Co-Design for CXL-Native Memory Tiering. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE. https://doi.org/10.1109/MICRO61859.2024.00111

[75] Zhiting Zhu, Newton Ni, Yibo Huang, Yan Sun, Zhipeng Jia, Nam Sung Kim, and Emmett Witchel. 2024. Lupin: Tolerating Partial Failures in a CXL Pod. In *Proceedings of the 2nd Workshop on Disruptive Memory Systems* (Austin, TX, USA) *(DIMES '24)*. Association for Computing Machinery, New York, NY, USA, 41–50. https://doi.org/10.1145/3698783.3699377